

Comparative Analysis of Process Scheduling Algorithm using AI models

Md. Moynul Asik Moni^{*}, Maharshi Niloy[†], Aquibul Haq Chowdhury[‡], Farah Jasmin Khan[§],
Md. Fahmid-Ul-Alam Juboraj[¶], Amitabha Chakrabarty^{||}

Department of Computer Science and Engineering
Brac University, Dhaka, Bangladesh

Email: ^{*}md.moynul.asik.moni@g.bracu.ac.bd, [†]maharshi.niloy@g.bracu.ac.bd, [‡]aquibul.haq.chowdhury@g.bracu.ac.bd,
[§]farah.jasmin.khan@g.bracu.ac.bd, [¶]md.fahmid.ul.alam.juboraj@g.bracu.ac.bd, ^{||}amitabha@bracu.ac.bd

Abstract—Process scheduling is an integral part of operating systems. The most widely used scheduling algorithm in operating systems is Round Robin, but the average waiting time in RR is often quite long. The purpose of this study is to propose a new algorithm to minimize waiting time and process starvation by determining the optimal time quantum by predicting CPU burst time. For burst time prediction, we are using the machine learning algorithms like linear regression, decision tree, k-nearest neighbors, and Neural Network Model Multi-Layer Perceptron. Moreover, for 10000 predicted burst time of processes with the same configuration, we have compared the average turnaround time, the average waiting time and the number of context switches of the proposed modified round robin algorithm with Traditional Round Robin, Modified Round Robin, Optimized Round Robin and Self-Adjustment Round Robin. The proposed modified round robin i.e. Absolute Difference Based Time Quantum Round Robin (ADRR) is found to be almost 2 times faster than the other algorithm in terms of process scheduling for the used dataset which contains a huge load of processes.

Index Terms—Average Turnaround Time, Average Waiting Time, Context Switch, Self-Adjustment Round Robin, Absolute Difference Based Time Quantum Round Robin, Optimized Round Robin

I. INTRODUCTION

In a modern multitasking operating system, efficiency is paramount. Therefore, a good scheduling algorithm is necessary to maximize computing efficiency. These algorithms can be judged on many criteria, such as CPU utilization, throughput, turnaround time (TT), waiting time (WT), response time, etc. Two scheduling algorithms of interest are shortest-remaining-time-first (SRTF) and round-robin (RR). In SRTF, each process is associated with its next CPU burst time, and the process with the smallest next CPU burst is chosen for execution. In RR, a small unit of time called the time quantum is used. The first process of the ready queue is allotted at most of the time quantum for execution then preempted and deposited at the tail of the ready queue. For a small quantum, there will be many context switches. If the time quantum is too large, RR will devolve into a first-come-first-serve (FCFS) algorithm [6].

We tried to improve the efficiency of the the processor based on RR and made an comparative analysis with the different artificial intelligence algorithms which includes decision tree (DT), multi-layer perceptron (MLP) linear regression (LR),

and k-nearest neighbours (KNN). If it is possible to train the processor to identify and predict the burst time accurately, it will be very much effective to calculate the time quantum more efficiently. So the comparison was done with different algorithms like self-adjustment round-robin (SARR), modified round-robin (MRRA) and optimized round-robin (ORR) after predicting the burst time with different algorithms and the same was done with our proposed algorithm where the time quantum was made dynamic. The result shows that our algorithm and the analytical process gives complete upper hand in decreasing the TT, WT, and reducing the context switches (CS) thus making the processors and CPU scheduling algorithms efficient and optimized. The machine learning models like DT [10] is a very useful tool in classification. KNN [7] is a widely used ML tool. Another powerful tool in machine learning (ML) is LR [1]. Here, we propose a new process scheduling algorithm and use ML and Neural Network (NN) models like DT, KNN, MLP and LR to determine the optimal time quantum dynamically.

The process scheduling algorithm is one of the most vital aspects of ensuring overall efficiency. With the advent of the operating systems and the processor, there have been a lot of algorithms and processor-level modifications to ensure the maximum utilization of the processor and operating system with minimum resource cost. Still, at present most of the algorithms we formulate or micro-level modifications are based on educated assumptions, and those are not exactly correct, because of different processes in different hardware systems and in different computers like cloud, grid [3], high performance, macro or micro along with different operating systems like Mac, Windows, Linux [2] or different subsystems the process scheduling is very much unpredictable, dynamic and nearly impossible. As a result, it gives rise to different problems like starvation of process, context overhead switching, delay of processor, deadlock, and process and resource allocation, aging, inefficient usage of CPU resources, wastage of internal memory, job pool queuing, etc. Also, another problem is the lack of analytical data based on different CPU scheduling algorithms and parameters like burst time, turnaround time, waiting time, and the estimated and most optimized time slice in the round-robin algorithm. So all these problems can be solved if the CPU can be made aware of the data or if we can

implement our algorithm with a predictive approach to make intelligent decisions.

We make the following contributions in this research.

- 1) We propose a new CPU scheduling algorithm.
- 2) We distinguish the performance between the proposed algorithm and the existing algorithms based on predicted CPU burst time.

II. RELATED WORK

In paper, [12], a new type of algorithm MARR (Median Average Round Robin) is proposed where the time quantum is dynamically changed. Here, the burst time of the new ready queue time quantum is calculated. The time quantum is set out to be the running burst time if it is empty, but if the time quantum isn't empty the average of the burst time is taken. It is seen that a new MARR (Median Average Round Robin) is better than other algorithms like RR, AMRR, AN, MMRR in average TT & AWT average WT but the number of CS is the same. It would have been more efficient if the context switch had been reduced.

In the paper, [9] the authors claimed a novel approach in which they used some Machine Learning methodologies dynamically to find out the most efficient CPU scheduling algorithm. Through experimentation, they showed that the overall performance of the DT in terms of accuracy and computational time tool setter than the other ML techniques.

Paper [5] has proposed a median-based modified Round Robin CPU scheduling algorithm that specifies the time quantum dynamically in each round depending on the process bursts time available in the ready queue. Here, they have worked with five processes with their proposed algorithm and compared the resulting parameters like the required number of context switching, average waiting time, average turnaround time, etc. to the existing available algorithms. They mostly focused on dynamic allocation of the time slice which is called time quantum for RR. Their algorithm illustrates a slightly modified RR algorithm where they approximate this time quantum in each round depending on the process available in the ready queue. In the paper [4], a new algorithm named Modified Median round Robin algorithm (MMRRA) where the time quantum is calculated through the root of the multiplication of the median and highest burst time. This algorithm is tested with the SJF algorithm where processes will go to the second round if the first round time quantum is calculated and the outburst time becomes greater than twenty percent of the total burst time; if not, the process will be completed by the CPU. When compared with other algorithms it has provided better results.

In the paper, [8], the proposed work upholds a dynamically modified RR algorithm based on K-Means clustering to consolidate the advent of favor 4 short processes and low scheduling overhead to minimize the average WT, average TT, and the number of CS. They first constructed some clusters depending on the process parameters using the K-Means algorithm. A method named Silhouette, they have used here for the evaluation of the processing cluster where it finds out

how effectively each data point lies within its cluster. A high average silhouette width indicates good clustering. Then they approximated a time slice for RR which directly contributes to the minimization of average WT and average TT. Using this pre-calculated time quantum value they again adjusted the time slice value depending on residual burst time. Surprisingly the proposed approach performs better than other algorithms proposed before in terms of efficient CPU scheduling.

In the paper [11], the Highest Response Ratio Next algorithm (HRRN) is a new algorithm where according to approximate run time, waiting time for each job is prioritized. The longer a job is starved the higher the priority it gets. Longer jobs with shorter life spans are ignored with the starved jobs. According to the largest response ratio, the CPU is given the work of a process. Priority is set by the summation of waiting & estimated time runtime divided by the estimated burst time. It was seen through tests that the average WT of HRRN was less than the traditional RR due to HRRN's lack of time quantum usage.

III. METHODOLOGY

A. Workflow

The purpose of the proposed algorithm (ADRR) is to minimize the starvation of processes for an extended period. We are going to do that by predicting the burst time of the processes with ML models, MLP, and exponential average. After predicting the burst time, we calculate the time quantum. The scheduling algorithm is going to schedule the processes with the calculated time quantum.

In this paper, we will use a DT, KNN, and LR as our ML models and MLP as a Neural Network model to predict the burst/run time from the collected dataset. There are a few stages to complete the training of the model:

- 1) Input Data Preprocessing: In this stage, the dataset is cleaned and formatted in such a way that makes the models learn and generalize the data better.
- 2) Processing: After splitting the dataset into training and testing, the training data is used to train the models.
- 3) Prediction: The testing data is used to make predictions and find the accuracy.

B. Data Collection

The input data was collected from the GWA-T-4 AuverGrid dataset. AuverGrid is a production grid platform where there are five clusters geographically located in the AuverGrid region of France. Each cluster has dual 3GHz Pentium-IV Xenons nodes running Scientific Linux. The total number of CPUs is 475 and the number of users is 405. Moreover, the number of jobs in the dataset is traced to be 404176. This dataset and its detailed analysis can be found at <http://gwa.ewi.tudelft.nl/datasets/gwa-t-4-auvergrid>. The dimension of the dataset is 404176x29.

C. Data Preprocessing

Initially, the raw dataset contains 29 columns of which the last 11 columns contain null values which were averaged.

Moreover, the average column called ‘ReqNProcs’ has one unique value. So, this column would not help in the learning process. So, we discard this column. We drop rows containing null values in ‘ReqMemory’ column. Furthermore, machine learning models can not work with string values. So, all string values were encoded. We dropped all correlated columns as we can see from figure 1. Again, we find out which features are more significant in predicting the burst time among all the available features by feature importance measurement. We find the score for each feature using the linear regression model. The model is trained and tested with each feature to find scores for each corresponding feature. We only keep features whose score is larger than 0.1 and drop all the other columns. Finally, the resulting dataset had a dimension of 347611x9 where 8 columns were picked as attributes, and ‘RunTime’ was selected as the target column. We perform all our experiments on this resulting dataset.

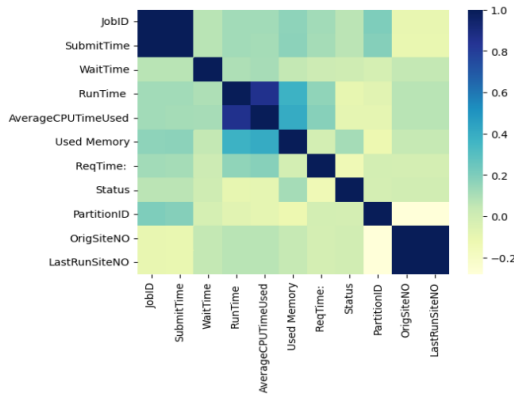


Fig. 1: Heatmap

USERID	0.015 196 729 510 913 443
LastRunSiteID	0.532 934 156 871 056 6
OrigSiteID	0.532 934 156 871 056 6
ExecutableID	0.058 746 315 829 638 63
GroupID	0.005 515 194 804 284 196
QueueID	0.002 129 171 205 123 015
PartitionID	0.545 594 011 774 175 3
Status	0.872 746 079 265 907
ReqMemory	0.055 970 465 669 952 23
ReqTime	2.566 266 196 019 828 5
ReqNProcs	6.161 160 028 828 05 $\times 10^{-5}$
Used Memory	14.046 796 027 826 481
AverageCPUTimeUsed	73.583 832 663 355 57
NProcs	6.161 160 028 828 05 $\times 10^{-5}$
WaitTime	0.958 956 287 390 266 8
SubmitTime	1.312 072 131 938 635 3
JobID	1.397 732 644 676 896 7

TABLE I: Feature Importance Scores

D. Implementation

This segment narrates the implementation of the proposed scheduling algorithm and the chosen ML and NN models for this work. The system that is used for this work, has the configuration - processor: AMD Ryzen 7 5800X, ram: 64

GB, GPU: Nvidia GeForce RTX 3060 and operating system: Windows 10.

1) *Burst time prediction*: Different models of ML and NN work in a different fashion for the prediction of the data. For getting appropriate predictions for this work primarily four models and a traditional prediction approach (exponential average) were tested.

2) *Comparative Analysis of Proposed Way and Other Existing Scheduling Algorithms via Running on the Predicted Burst Time*: At this point, the predicted burst times go to the proposed algorithm as well as to other existing and previously proposed algorithms for the execution to determine the 3 comparison parameters; average TT and average WT and CS. TT is the time that is needed to complete the execution of a process and WT is the time that is needed for a process to wait to accomplish the whole of its execution. A CS is a number that shows how many times the CPU is given to the processes throughout the operation. On the basis of these 3 parameters, the efficiency of the proposed way has been estimated. Algorithm 1 shows the pseudo-code of the proposed way. The predicted burst times from the decision tree model along with their process ids and priorities go as input to the proposed way as well as in RR, SARR, MRRA and ORR. And from this section 3 parameters, average TT and average WT and the number of the CS are deduced for the comparison. The Existing Models are previously proposed by:

- 1) Traditional Round Robin [Existing]
- 2) Modified Round Robin Algorithm (MRRA) (Pradhan and Ray, 2016)
- 3) Self-Adjustment Round Robin (SARR) (Matarneh and Rami, 2009)
- 4) Optimized Round Robin (ORR) (Biswas and Saha, 2018)

IV. RESULTS AND ANALYSIS

A. Accuracy analysis of burst time prediction

1) Exponential Average:

$$T_{n+1} = \alpha t_n + (1 - \alpha)T_n \quad (1)$$

where, α = smoothing factor and $0 \leq \alpha \leq 1$, t_n = actual burst time of nth process and T_n = predicted burst time of nth process. This method scores about 26.08% of accuracy for the burst time prediction on the chosen data-set.

2) *Linear Regression (LR)*: This model has performed quite well since the data-set is continuous in fashion as well as there is no cluster. The actual accuracy score of the LR model was more than 97.96% for this work in terms of predicting burst time. Figure 2a shows the train and validation accuracy and figure 2b shows the train and validation loss of the linear regression model for this work.

The below figure 3 highlights the scatter plotting of the actual burst time and the predicted burst time for the LR Model.

function ADDR (*burst_time*, *process_id*, *arrival_time*, *priority_list*)

```

    process_list ← {};
    previous_time_quantum ← 0;
    flag ← True;
    for e ∈ burst_time do add process p with
        associated process_id in process_list;
    while process_list ≠ ∅ do
        for new_process ∈ burst_time do add
            process p with associated process_id in
            process_list;
        Sort (process_list);
        if priority_list ≠ ∅ then add priority process
            in front of the process_list with process_id;
        time_quantum ← Abs (process_list.values () [-1] -
            process_list.values () [0]);
        if time_quantum = 0 and flag = True then
            time_quantum ←
                process_list.values () [-1];
            flag = False;
        end
        if time_quantum = 0 then
            time_quantum ←
                previous_time_quantum;
        for p ∈ process_list do
            provide CPU to process p for
                time_quantum unit of time;
            if process execution is done then
                remove it from process_list;
                if p ∈ priority_list then remove it
                    from priority_list;
            else
                decrease burst_time of p for
                    time_quantum unit from
                    process_list;
            end
        end
    end
end
end

```

Algorithm 1: Pseudo-code of ADDR

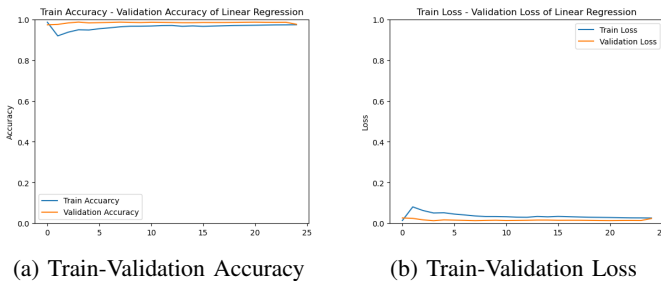


Fig. 2: Train-Validation Accuracy and Loss of LR

3) *K-Nearest Neighbor (KNN)*: This model has not performed well on our data-set, KNN has scored approximately 17%. Figure 4a shows the train and validation accuracy and

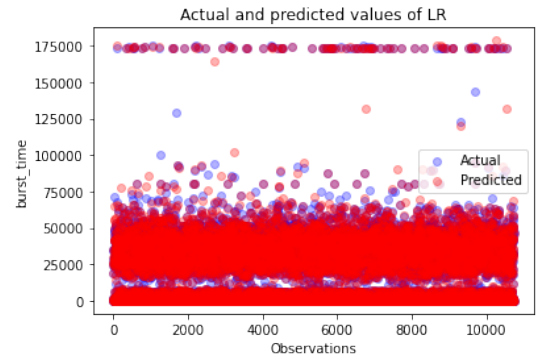


Fig. 3: Actual-Predicted Data Points of LR

figure 4b shows the train and validation loss of the KNN model for this work.

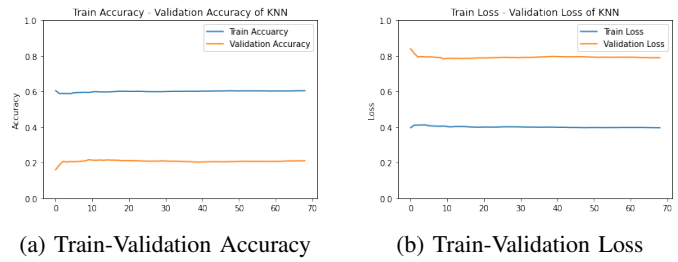


Fig. 4: Train-Validation Accuracy and Loss of KNN

The below figure 5 highlights the scatter plotting of the actual burst time and the predicted burst time for the KNN Model.

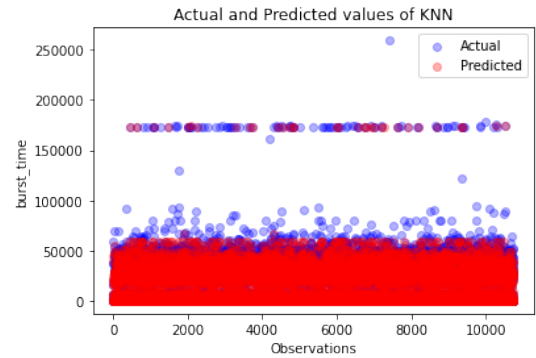


Fig. 5: Actual-Predicted Data Points of KNN

4) *Neural Network (NN)*: NN (MLP- with 20 hidden layers, 0.001 learning rate) has scored about 26.01% for our data-set.

5) *Decision Tree (DT)*: has performed quite well over other ML/NN algorithms on our data-set, the actual accuracy score of this model was more than 98.64% for this work in terms of predicting burst time. Figure 6a shows the training and validation accuracy and figure 6b shows the training and validation loss of the decision tree model for this work.

Figure 7 highlights the scatter plotting of the actual burst time and the predicted burst time for the Decision Tree Model.

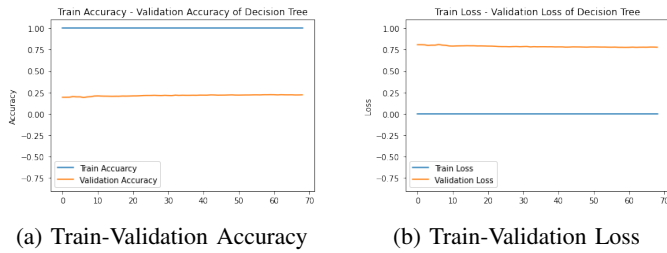


Fig. 6: Train-Validation Accuracy and Loss of DT

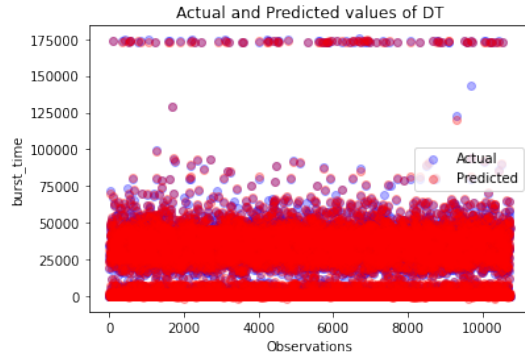


Fig. 7: Actual-Predicted Data Points of DT

Table II shows the prediction accuracy score of the used exp avg and used ML/NN models.

Model Name	Accuracy
Exponential AVG	26.08%
K-Nearest Neighbor (KNN)	17.1%
Linear Regression (LR)	97.96%
Decision Tree (DT)	98.64%
Neural Network (MLP)	26.01%

TABLE II: Score of Exponential Average, KNN, LR, DT, and NN.

B. Scheduling efficiency analysis of proposed algorithm with others

For the measurement of the efficiency of the proposed algorithm (ADRR), 10000 processes were put as input for both the proposed method (ADRR) and RR, SARR, MRRA and ORR algorithm where a general processor is given 10 to 200 processes at a time by the OS. Five algorithms run in the same environment and under the same configuration. The final outcome points out that the proposed algorithm (ADRR) ensures almost half of the TT and the WT is less than the other 4. Also, it ensures less number of CS. The below Table III shows the resultant output of the 5 algorithms and consecutively shows the comparison of the average WT, the number of CS and the average TT of the proposed algorithms to the others. From the table III, it is clear that the proposed algorithm (ADRR) is almost 2 times faster than the other existing and proposed algorithm (ADRR) in terms of process scheduling under a huge load of processes. Also, it can be inferred that although the sorting mechanism of the proposed

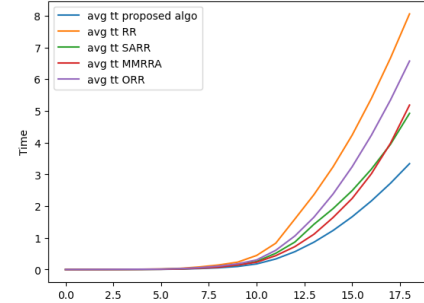
algorithm creates some time complexity, the high performance of the algorithm makes this time complexity negligible.

Algorithm	Average Turnaround Time (avg AT)	Average Waiting Time (avg WT)	Number of Context Switch (CS)
Proposed Way (ADRR)	40331930.48	40312117.96	20002
Traditional Round Robin (RR)	87194390.98	87174578.46	28964
Self-Adjustment Round Robin (SARR)	72398064.70	72378252.18	39956
Modified Round Robin Algorithm (MRRA)	84924105.36	84904292.84	25208
Optimized Round Robin (ORR)	78508779.73	78488967.20	22470

TABLE III: Comparison of the CPU Scheduling Algorithms

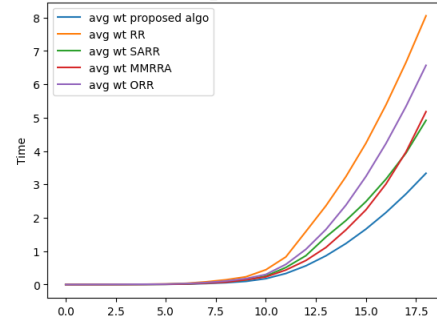
For better illustration, the proposed algorithm (ADRR) and the other 4 have run on 5000 processes depending upon 10 consecutive epochs wherein each epoch the process list gets 500 more randomized processes than the previous step. Here Figure 8a and 8b illustrates the outcomes of this extensive consecutive test in terms of average TT and average WT. From the figures, it is clear that the proposed algorithm always took less time in terms of average TT and average WT than the other algorithms in every situation whether it is a small batch of processes or a large batch of processes.

Average Turnaround time of Proposed_way vs. other Algorithms (consecutive test)



(a) Average Turnaround Time of Consecutive Test

Average Waiting time of Proposed_way vs. other Algorithms (consecutive test)



(b) Average Waiting Time of Consecutive Test

Fig. 8: Consecutive Test

V. CONCLUSION

This work will contribute by adding intelligence to the operating system end level of the processor or at least it will be possible to provide necessary information to the processor to take decisions intelligently from the user/software level for the overall optimization. Since we are proposing a new algorithm along with machine learning connectivity for the process scheduling algorithm and evaluating the work done previously. This proposal can be further extended to improved scheduling algorithm implementation along with extending the horizon for new domains in AI and OS. Implementation of learning with the operating system itself is a unique domain and our research will pave the way for future researchers who will be working on this domain.

In the future, we would like to continue this research by implementing other AI models and on other different datasets via real implementation of the operating system end.

REFERENCES

- [1] J. M. Stanton, "Galton, pearson, and the peas: A brief history of linear regression for statistics instructors," en, *Journal of Statistics Education*, vol. 9, no. 3, p. 3, Jan. 2001, ISSN: 1069-1898. DOI: 10.1080/10691898.2001.11910537. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/10691898.2001.11910537>.
- [2] K. K. P. and A. Negi, "Characterizing process execution behavior using machine learning techniques," in *In DpROMWorkShop Proceedings, HiPC 2004 International Conference*, 2004.
- [3] T. Helmy, S. Al-Azani, and O. Bin-Obaidellah, "A machine learning-based approach to estimate the cpu-burst time for processes in the computational grids," in *2015 3rd International Conference on Artificial Intelligence, Modelling and Simulation (AIMS)*, Kota Kinabalu, Malaysia: IEEE, Dec. 2015, pp. 3–8, ISBN: 9781467386753. DOI: 10.1109/AIMS.2015.11. [Online]. Available: <http://ieeexplore.ieee.org/document/7604542/>.
- [4] H. Mora, S. E. Abdullahi, and S. B. Junaidu, "Modified median round robin algorithm (MMRRA)," in *2017 13th International Conference on Electronics, Computer and Computation (ICECCO)*, Abuja, Nigeria: IEEE, Nov. 2017, pp. 1–7, ISBN: 9781538624999. DOI: 10.1109/ICECCO.2017.8333325. [Online]. Available: <http://ieeexplore.ieee.org/document/8333325/>.
- [5] M. Tajwar, M. Pathan, L. Hussaini, and A. Abubakar, "Cpu scheduling with a round robin algorithm based on an effective time slice," *Journal of Information Processing Systems*, vol. 13, Jan. 2017. DOI: 10.3745/JIPS.01.0018.
- [6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating system concepts*, eng, Tenth edition. Hoboken, NJ: Wiley, 2018, ISBN: 9781119439257.
- [7] H. Agarwal and G. Jariwala, "Analysis of process scheduling using neural network in operating system," in Jan. 2020, pp. 1003–1014, ISBN: 978-981-15-0145-6. DOI: 10.1007/978-981-15-0146-3_97.
- [8] S. M. Mostafa and H. Amano, "Dynamic round robin CPU scheduling algorithm based on k-means clustering technique," en, *Applied Sciences*, vol. 10, no. 15, p. 5134, Jul. 2020, ISSN: 2076-3417. DOI: 10.3390/app10155134. [Online]. Available: <https://www.mdpi.com/2076-3417/10/15/5134>.
- [9] S. Tehsin, Y. Asfia, N. Akbar, F. Riaz, S. Rehman, and R. C. D. Young, "Selection of cpu scheduling dynamically through machine learning," in *Pattern Recognition and Tracking XXXI*, M. S. Alam, Ed., Online Only, United States: SPIE, Apr. 2020, p. 24, ISBN: 9781510635777. DOI: 10.1117/12.2559540. [Online]. Available: <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11400/2559540/Selection-of-CPU-scheduling-dynamically-through-machine-learning/10.1117/12.2559540.full>.
- [10] Z. Wang, X. Zhang, and G. Peng, "An improved integrated scheduling algorithm with process sequence time-selective strategy," en, *Complexity*, vol. 2021, A. E.-B. Hassanien, Ed., pp. 1–10, Mar. 2021, ISSN: 1099-0526, 1076-2787. DOI: 10.1155/2021/5570575. [Online]. Available: <https://www.hindawi.com/journals/complexity/2021/5570575/>.
- [11] B. Richardson and W. Istiono, "Comparison analysis of round robin algorithm with highest response ratio next algorithm for job scheduling problems," *International Journal of Open Information Technologies*, vol. 10, no. 2, pp. 21–26, 2022.
- [12] Sakshi, C. Sharma, S. Sharma, et al., "A new median-average round robin scheduling algorithm: An optimal approach for reducing turnaround and waiting time," en, *Alexandria Engineering Journal*, vol. 61, no. 12, pp. 10527–10538, Dec. 2022, ISSN: 11100168. DOI: 10.1016/j.aej.2022.04.006. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1110016822002599>.